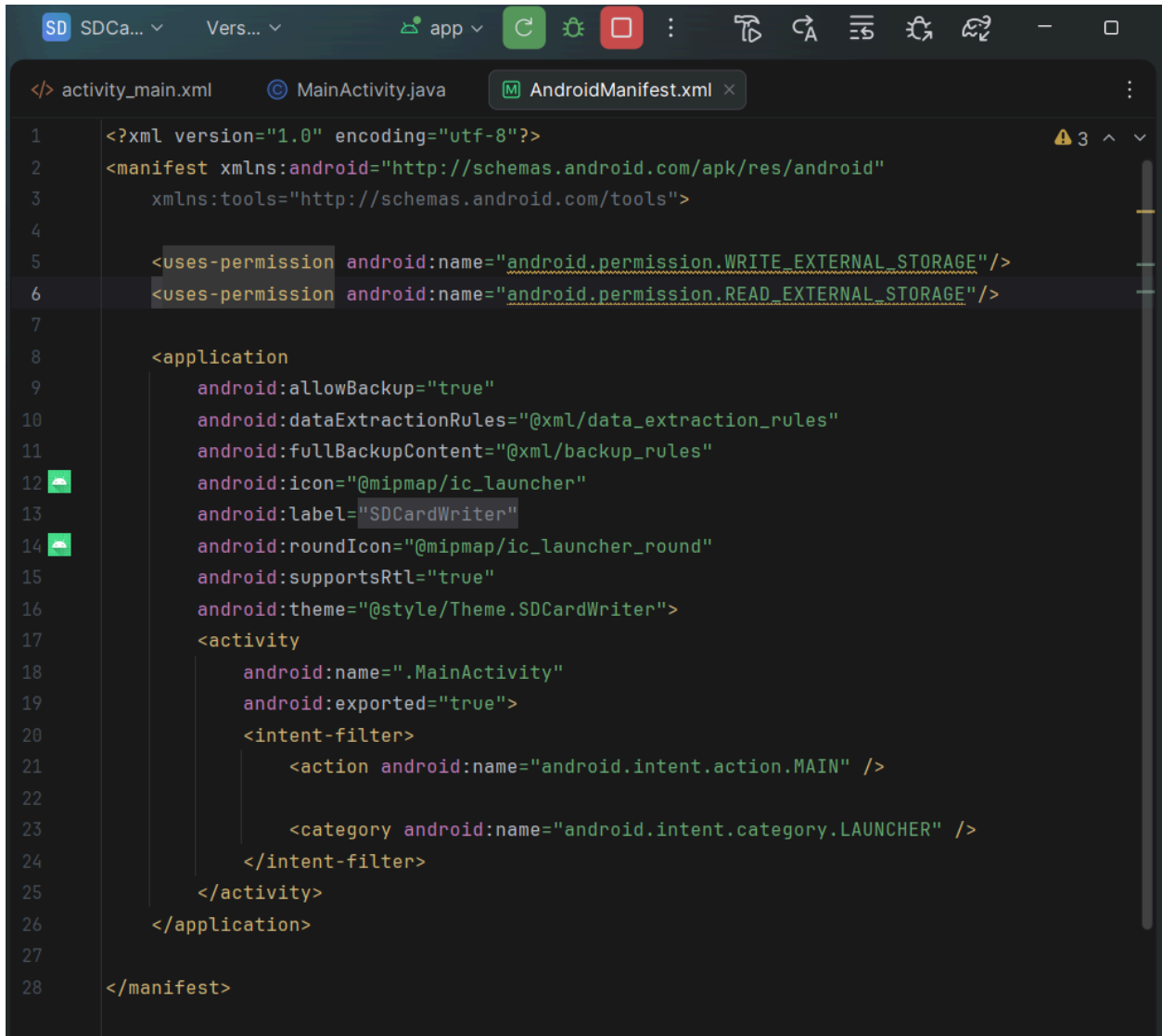


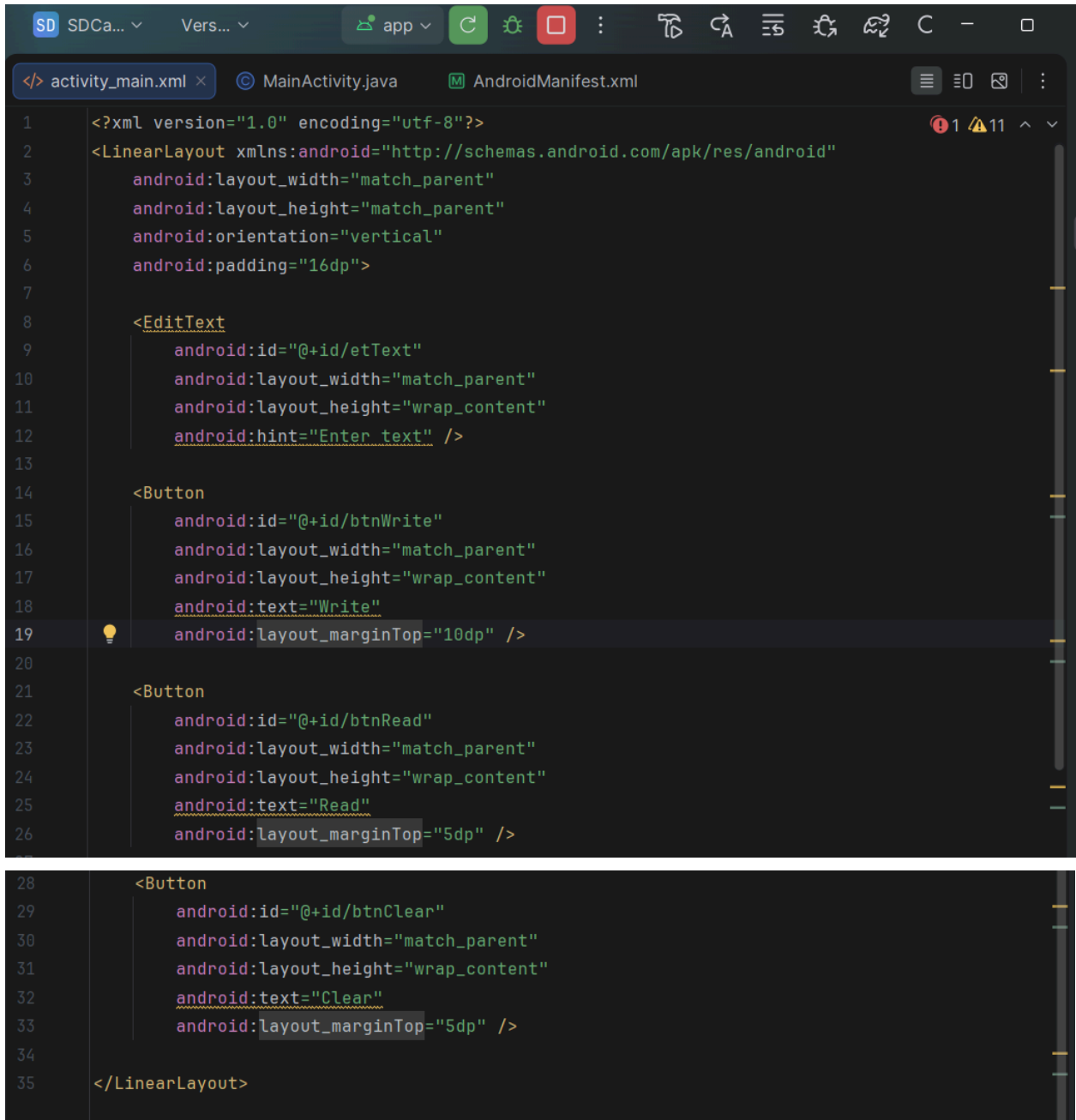
Program:

→AndroidManifest.xml



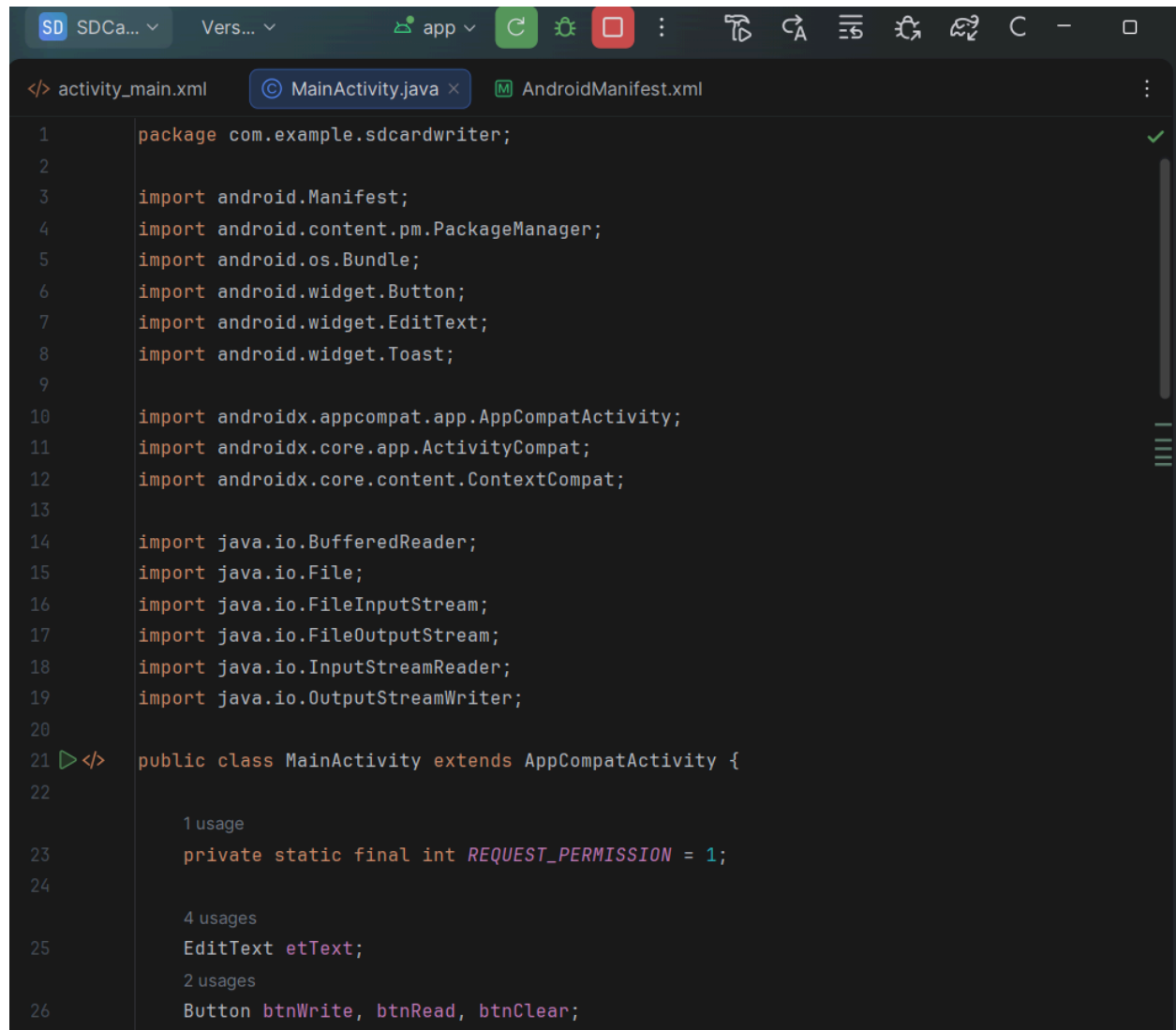
```
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:tools="http://schemas.android.com/tools">
4
5     <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
6     <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
7
8     <application
9         android:allowBackup="true"
10        android:dataExtractionRules="@xml/data_extraction_rules"
11        android:fullBackupContent="@xml/backup_rules"
12        android:icon="@mipmap/ic_launcher"
13        android:label="SDCardWriter"
14        android:roundIcon="@mipmap/ic_launcher_round"
15        android:supportsRtl="true"
16        android:theme="@style/Theme.SDCardWriter">
17        <activity
18            android:name=".MainActivity"
19            android:exported="true">
20            <intent-filter>
21                <action android:name="android.intent.action.MAIN" />
22
23                <category android:name="android.intent.category.LAUNCHER" />
24            </intent-filter>
25        </activity>
26    </application>
27
28 </manifest>
```

→activity\_main.xml



```
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      android:layout_width="match_parent"
4      android:layout_height="match_parent"
5      android:orientation="vertical"
6      android:padding="16dp">
7
8      <EditText
9          android:id="@+id/etText"
10         android:layout_width="match_parent"
11         android:layout_height="wrap_content"
12         android:hint="Enter text" />
13
14     <Button
15         android:id="@+id/btnWrite"
16         android:layout_width="match_parent"
17         android:layout_height="wrap_content"
18         android:text="Write"
19         android:layout_marginTop="10dp" />
20
21     <Button
22         android:id="@+id/btnRead"
23         android:layout_width="match_parent"
24         android:layout_height="wrap_content"
25         android:text="Read"
26         android:layout_marginTop="5dp" />
27
28     <Button
29         android:id="@+id/btnClear"
30         android:layout_width="match_parent"
31         android:layout_height="wrap_content"
32         android:text="Clear"
33         android:layout_marginTop="5dp" />
34
35 </LinearLayout>
```

→MainActivity.java

The image shows a screenshot of an IDE window with three tabs: 'activity\_main.xml', 'MainActivity.java', and 'AndroidManifest.xml'. The 'MainActivity.java' tab is active, displaying Java code for an Android application. The code includes package declarations, imports for Android and Java classes, and the definition of the MainActivity class which extends AppCompatActivity. The IDE interface includes a top toolbar with various icons for file operations and a right sidebar with icons for tool windows. Line numbers are visible on the left side of the code editor.

```
1 package com.example.sdcardwriter;
2
3 import android.Manifest;
4 import android.content.pm.PackageManager;
5 import android.os.Bundle;
6 import android.widget.Button;
7 import android.widget.EditText;
8 import android.widget.Toast;
9
10 import androidx.appcompat.app.AppCompatActivity;
11 import androidx.core.app.ActivityCompat;
12 import androidx.core.content.ContextCompat;
13
14 import java.io.BufferedReader;
15 import java.io.File;
16 import java.io.FileInputStream;
17 import java.io.FileOutputStream;
18 import java.io.InputStreamReader;
19 import java.io.OutputStreamWriter;
20
21 </> public class MainActivity extends AppCompatActivity {
22
23     1 usage
24     private static final int REQUEST_PERMISSION = 1;
25
26     4 usages
27     EditText etText;
28
29     2 usages
30     Button btnWrite, btnRead, btnClear;
```

```
27
28
29 @Override
protected void onCreate(Bundle savedInstanceState) {
30     super.onCreate(savedInstanceState);
31     setContentView(R.layout.activity_main);
32
33     etText = findViewById(R.id.etText);
34     btnWrite = findViewById(R.id.btnWrite);
35     btnRead = findViewById(R.id.btnRead);
36     btnClear = findViewById(R.id.btnClear);
37
38     // Runtime permission (API 23+)
39     if (ContextCompat.checkSelfPermission(context: this,
40         Manifest.permission.WRITE_EXTERNAL_STORAGE)
41         != PackageManager.PERMISSION_GRANTED) {
42
43         ActivityCompat.requestPermissions(activity: this,
44             new String[]{Manifest.permission.WRITE_EXTERNAL_STORAGE,
45                 Manifest.permission.READ_EXTERNAL_STORAGE},
46                 REQUEST_PERMISSION);
47     }
48
49     btnWrite.setOnClickListener(View v -> writeToFile());
50     btnRead.setOnClickListener(View v -> readFromFile());
51     btnClear.setOnClickListener(View v -> etText.setText(""));
52 }
53
1 usage
```

```

53
54 1 usage
55 private void writeToFile() {
56     String text = etText.getText().toString();
57
58     if (text.isEmpty()) {
59         Toast.makeText(context: this, text: "Enter text first", Toast.LENGTH_SHORT).show();
60         return;
61     }
62
63     try {
64         File file = new File(getExternalFilesDir( type: null), child: "sdcard_data.txt");
65
66         FileOutputStream fOut = new FileOutputStream(file);
67         OutputStreamWriter writer = new OutputStreamWriter(fOut);
68
69         writer.write(text);
70         writer.close();
71         fOut.close();
72
73         Toast.makeText(context: this, text: "Data written to SD card", Toast.LENGTH_SHORT).show();
74     } catch (Exception e) {
75         Toast.makeText(context: this, text: "Error: " + e.getMessage(), Toast.LENGTH_SHORT).show();
76     }
77 }
78

```

```

79
80 1 usage
81 private void readFromFile() {
82     try {
83         File file = new File(getExternalFilesDir( type: null), child: "sdcard_data.txt");
84
85         FileInputStream fIn = new FileInputStream(file);
86         BufferedReader reader = new BufferedReader(new InputStreamReader(fIn));
87
88         StringBuilder buffer = new StringBuilder();
89         String line;
90
91         while ((line = reader.readLine()) != null) {
92             buffer.append(line).append("\n");
93         }
94
95         reader.close();
96
97         etText.setText(buffer.toString());
98         Toast.makeText(context: this, text: "Data read from SD card", Toast.LENGTH_SHORT).show();
99     } catch (Exception e) {
100         Toast.makeText(context: this, text: "Error reading file: " + e.getMessage(), Toast.LENGTH_SHORT).show();
101     }
102 }

```

OUTPUT:

